GPU terminology cheat sheet

NVIDIA, AMD, Intel and Apple. A shared vocabulary for GPU compute.

Concept	NVIDIA CUDA	AMD HIP	Intel Xe + SYCL	Apple Metal	Shared meaning Common role or scope; hardware and API details vary
01 **Execution**	How the work is divided				
Launch grid	Grid	Grid	ND-range	Grid	The full set of threads or work-items in one kernel launch.
Cooperating threads	Thread block	Thread block Work-group in hardware docs	Work-group	Threadgroup	A group that can share memory and synchronize at a group barrier.
Execution subgroup	Warp	Wavefront Warp in HIP	Sub-group	SIMD-group	Threads grouped for instruction execution. Width and synchronization rules depend on the target.
One invocation	Thread	Thread	Work-item	Thread	One indexed execution of the kernel, with its own private state.
02 **Hardware**	Closest roles; hardware boundaries and counts differ				
Compute block	Streaming Multiprocessor SM	Compute Unit (CU) CDNA 1-4 example	Xe-core	Shader core	A hardware block that executes groups of threads. Size and resources differ across vendors.
03 **Memory**	Who owns the data, and where it lives				
Registers	Registers	Vector / scalar registers VGPRs / SGPRs	General Register File GRF	Registers	Storage for values used by executing instructions. Allocation and physical backing vary.
Group-shared memory	Shared memory	Local Data Share LDS	Shared Local Memory SLM	Threadgroup memory	Program-managed storage shared by threads in one cooperating group.
Private memory	Local memory	Scratch memory	Scratch memory	Thread stack / spills	Per-thread storage outside registers, used for spills, stack data and other private values.
Device-wide memory	Global memory	Global memory	Global memory	Device address space	Memory accessible to GPU threads across cooperating groups. Coordinate conflicting accesses to avoid data races.
Caches	L1 / L2 caches	Vector / scalar caches; L2 CDNA example	L1 / L2 caches Xe-HPC example	Shader-core caches; GPU last-level cache	Hardware-managed storage that reuses recently accessed data. Cache levels and sharing scopes differ.
04 **Coordination**	How lanes cooperate and the host submits work				
Group barrier	__syncthreads()	__syncthreads()	group_barrier(g) SYCL work-group	threadgroup_barrier(flags)	A rendezvous for cooperating threads. Follow the API's participation and memory-fence rules.
Lane exchange	__shfl_sync	__shfl	select_from_group SYCL sub-group	simd_shuffle	Read a value from another lane in the same execution subgroup.
Atomic addition	atomicAdd Example	atomicAdd Example	fetch_add sycl::atomic_ref member	atomic_fetch_add_explicit Example	Add to a scalar memory value indivisibly. Supported types, memory order and scope depend on the API.
Work submission	CUDA stream	HIP stream	SYCL queue	MTLCommandQueue MTL4CommandQueue	Submit GPU work through a stream or queue. Ordering and dependency rules differ.
Work dependencies	CUDA event	HIP event	SYCL event	MTLEvent MTLSharedEvent	Record or signal progress so dependent work can wait. Event models differ across APIs.
05 **Performance**	The same questions, with different counters				
Occupancy	Warp occupancy	Wave occupancy	Xe-core occupancy Hardware threads	Thread occupancy SIMD-groups	Resident execution state, measured as a count or share of capacity. Higher occupancy need not mean higher performance.
Divergence	Warp divergence	Wavefront divergence	SIMD divergence Within a sub-group	SIMD-group divergence	Threads in an execution subgroup take different code paths, which can reduce useful parallel work.

Memory names are easy to mix up

CUDA local memory is private to a thread. Intel SLM and Metal threadgroup memory are shared within a group. Metal private resource storage controls CPU access.

Hardware counts are not interchangeable

SM, CU, Xe-core and shader core describe different hardware organizations. Their counts do not measure equal amounts of compute.

Use the API's synchronization rules

A subgroup does not guarantee implicit synchronization. Barriers, shuffles, atomics and queues each have participation and ordering rules.

Hardware examples: AMD CDNA 1-4, Intel Xe family, Apple M3 / A17 Pro. Organization and caches vary by generation. SYCL is vendor-neutral.

Sources **NVIDIA CUDA** **AMD HIP / CDNA** **Intel Xe** **Apple Metal** **Khronos SYCL**

32 references / Checked 20 Oct 2026

Per-term citations: gpu-terminology-sources.html